Module 06 — Testing Data Quality

Tier: 🟢 Beginner · **Duration:** 90 min · **Prerequisites:** Module 05

> ***Framing:*** *Testing isn't optional. CI rejects any Silver or Gold PR that's missing mandatory tests on key columns. This module sets that expectation from the start and treats* `dbt build` *as the only acceptable command for running models and tests together.*

Agenda

Time	Duration	Topic	Learning Goal	Mode	Participant Activity	Materials	Tr
00:00	10 min	Recap Module 05	Confirm sources mental model	Q&A	Answer from memory	—	Ask all 4 pre "what colun freshness?"
00:10	10 min	Why tests exist — and why they're mandatory	Understand what tests protect and what happens without them	Present	Listen	This doc	Use a real s rename in B staging mo catches it u wrong two
00:20	15 min	Two test types	Know generic vs. singular and when to use each	Present + live code	Annotate examples	This doc	Emphasise: 95% of case for complex one of each
00:35	20 min	The four generic tests	Write correct test YAML for all four	Present + live code	Follow along in editor	This doc	Write tests l `select dim_` and fail out break a `not_` failure mess
00:55	5 min	Test severity: `warn` vs. `error`	Know when to use each	Present	Annotate	This doc	One rule: `er` and FK relat checks like ratios.
01:00	5 min	`dbt build` — the only correct command	Understand why `dbt build` is mandatory in CI	Present	—	This doc	State this cl `test` is wron can fail afte tests still ru first failure.
01:05	5 min	Mandatory test requirements	Know exactly	Present	Write down requirements	This doc	Read these These are n will reject P

			what CI enforces				
01:10	30 min	Exercise: add tests, break them, read the output	Write tests, run them, interpret failure messages	Practice	Solo exercise	Exercise below	Circulate. Ke write test YA and read fai trainer help
01:40	10 min	Debrief + prep questions	Consolidate	Debrief	Verbal	—	Ask: "what v dashboards `fct_prescrip` had duplica caught it?" -

---

Content

Part A — Why Tests Are Mandatory

Here's a scenario that happens more often than you'd think. Someone renames a column in Bronze. The staging model that depended on that column silently breaks. No test catches it. Two weeks later, a dashboard shows wrong numbers and the business reports it.

Without tests:

- A column rename in Bronze breaks a staging model — silently
- Duplicate surrogate keys corrupt Power BI relationships — silently
- A NULL value in a required column propagates downstream — silently
- A Gold mart shows wrong numbers — and nobody knows until the business reports it

Tests are a CI requirement for Silver and Gold models. A PR without mandatory tests on key columns won't be merged.

Part B — Two Types of dbt Tests

1. Generic tests — 95% of your tests

You define these in `schema.yml`. They're reusable and apply to columns with minimal YAML.

```yaml
models:
  - name: fct_prescription
    columns:
      - name: prescription_key
        tests:
          - unique
          - not_null
```

2. Singular tests — for complex business rules

These are standalone `.sql` files in the `tests/` folder.

> ***They return rows on failure*** *— no rows means the test passes. The most common pattern is a* `GROUP BY ... HAVING` *query that returns rows violating a business rule:*

```sql
-- tests/assert_fct_revenue_no_negative_amounts.sql
-- Fails if any combination of keys produces a negative net revenue

SELECT
    customer_key,
    product_key,
    SUM(amount_net) AS total_amount_net
FROM {{ ref('fct_revenue') }}
GROUP BY 1, 2
HAVING total_amount_net < 0
```

Use this pattern for rules that operate on aggregations — things `not_null` and `unique` can't express. The second common pattern uses `LEFT JOIN ... WHERE IS NULL` for referential checks that cross multiple models:

```sql
-- tests/assert_no_orphan_prescriptions.sql
-- Fails if any prescription has no matching patient

SELECT p.prescription_key
FROM {{ ref('fct_prescription') }} p
LEFT JOIN {{ ref('dim_patient') }} d
    ON p.patient_key = d.patient_key
WHERE d.patient_key IS NULL
```

Use singular tests when the rule can't be expressed in YAML. They're harder to read, so prefer generic tests whenever possible.

Part C — The Four Generic Tests

`unique`

```yaml
- name: prescription_key
  tests:
    - unique
```

This fails if any value appears more than once. It's required on all `_key` columns in Silver and Gold.

`not_null`

```yaml
- name: prescription_key
  tests:
    - not_null
```

This fails if any value is `NULL`. Required on all `_key` columns.

`accepted_values`

```yaml
- name: appointment_status
  tests:
```

```
    - accepted_values:
        values: ['scheduled', 'completed', 'cancelled', 'no_show']
```

This fails if any value outside the list appears. Use it for status columns, type columns, and flag columns.

`relationships`

```
- name: patient_key
  tests:
    - relationships:
        to: ref('dim_patient')
        field: patient_key
```

This fails if any `patient_key` in this model doesn't exist in `dim_patient`. It's a foreign key integrity check. It's required on all FK columns in Silver facts and Gold marts.

Part D — Test Severity

You can configure severity in two places: globally in `dbt_project.yml`, or per individual test in `schema.yml`.

Option 1 — Global default in `dbt_project.yml`

```
# dbt_project.yml
tests:
  analytics:           # project namespace
    +severity: warn    # default for every test in the project

    silver:
      +severity: error # Silver layer overrides — all tests here are errors

    gold:
      +severity: error # Gold layer overrides — all tests here are errors
```

This sets a project-wide default and lets you tighten it per layer. The example above is the pattern we use: `warn` as the safe default, `error` locked in for Silver and Gold where data quality is mandatory.

Option 2 — Per individual test in `schema.yml`

```
- name: prescription_key
  tests:
    - unique:
        config:
          severity: error    # CI fails, pipeline stops ← required on all _key columns
    - not_null:
        config:
          severity: error

- name: cancellation_note
  tests:
    - not_null:
```

```
      config:
        severity: warn     # CI logs a warning but does NOT stop the pipeline
```

Per-test config overrides any global setting from `dbt_project.yml`. Use this when a single test inside an otherwise strict layer needs to be relaxed.

Rule:

- `error` — all `_key` columns, all FK relationships, all required business columns
- `warn` — soft checks where some nulls are expected (free text fields, optional columns)

CI doesn't stop on `warn`. It does stop on `error`. Never use `warn` as a workaround for a test you don't want to fix.

Part E — `dbt build` Is the Only Correct Command

```
# ❌ WRONG — tests run even if a model failed; run and test are decoupled
dbt run && dbt test

# ✅ CORRECT — stops at the first failure; models and tests run in DAG order
dbt build
```

What `dbt build` does:

1. For each node in the DAG, in dependency order:
    - Run the model (or seed/snapshot)
    - Run its tests immediately after
    - If any test fails with `error` severity, stop — don't run downstream models

Why this matters: If `dim_patient` has a duplicate `patient_key` test failure, `dbt build` stops before running `fct_prescription`. With `dbt run && dbt test`, all models run first and all tests run after — by which time you've already loaded bad data into `fct_prescription`.

In CI: always `dbt build`. Locally: prefer `dbt build`. Only use `dbt test --select <model>` when you're debugging a specific failing test.

Part F — Mandatory Test Requirements

These are checked on every Silver and Gold PR:

Column type	Required tests
`_key` columns (surrogate PKs)	`unique` + `not_null` (both, always)
FK columns (referencing another model)	`relationships`
Status / type columns	`accepted_values` (if finite value set exists)
Business-critical measures	`not_null`

What CI checks automatically:

- Silver and Gold models must have at least one test per `_key` column

- A PR with zero tests on a new Silver model won't pass review

Reference: `dbt-test-strategy` skill — full test placement guide across Bronze/Silver/Gold.

Exercise (30 min)

> ***Project context:*** *The full staging layer is complete. Silver models exist but* `models/silver/schema.yml` *doesn't yet. This session creates it with a full test suite.*

Task 1 — Create `models/silver/schema.yml` with tests for `fct_prescription`

Create the file. Add a `version: 2` header and a `models:` block for `fct_prescription`.

The model has these columns:

Column	Type	Required test(s)	Severity
`prescription_key`	Surrogate PK	unique + not_null	error
`patient_key`	FK → `dim_patient.patient_key`	not_null + relationships	error
`doctor_key`	FK → `dim_doctor.doctor_key`	not_null + relationships	error
`prescription_date`	Date	not_null	error
`medication_type`	String	accepted_values: tablet, liquid, injection, topical	error
`dosage_amount`	Number	not_null	warn (nulls expected when dose unconfirmed)
`notes`	String	none	—

Task 2 — Run the tests

```
dbt test --select fct_prescription
```

All 7 tests pass on the clean dataset. Note which tests ran and under which model name they appear in the output.

Task 3 — Break a test deliberately

Insert a duplicate row in your dev schema:

```
INSERT INTO SILVER_DEV.TESTING__dev_yourname.fct_prescription
SELECT * FROM SILVER_DEV.TESTING__dev_yourname.fct_prescription
WHERE prescription_key = 'rx-001';
```

Run `dbt test --select fct_prescription` again. Read the failure output:

- Which test failed?
- How many rows were returned?

- Copy the test SQL from the output and run it manually in Snowsight.

Restore clean data:

```
dbt run --select fct_prescription
```

Task 4 — Write a singular test

Create `tests/assert_fct_prescription_no_zero_dosage.sql`.

Return rows where `dosage_amount = 0`. Zero is a data error — it's impossible to dispense nothing. Null is allowed (dose pending confirmation). If this query returns any rows, the test fails.

Run it:

```
dbt test --select test_type:singular
```

Task 5 — Run `dbt build` across the Silver layer

```
dbt build --select silver.*
```

Watch the DAG order. Each model is followed immediately by its tests. If `dim_patient` fails a test, `fct_prescription` — which depends on it — is never built.

▶ Expected schema.yml

▶ Expected singular test

Reference Material

- dbt testing docs
- dbt generic tests
- dbt test severity
- Internal: `dbt-test-strategy` skill — when and where to test per layer

Prep Questions for Module 07

1. What's the difference between a generic test and a singular test?
2. Name the four built-in generic tests.
3. What does `dbt build` do differently from `dbt run && dbt test`?
4. Which two tests are mandatory on every `_key` column in Silver?